

SALA DO FUTURO 2

Como organizar pastas, HTML, CSS e JavaScript

Objetivo: montar uma fundacao de codigo organizada antes de o projeto crescer.

Foco: estrutura de pastas, ordem de escrita, vinculacao de arquivos e boas praticas simples.

Nivel: base pratica para prototipo escolar e evolucao futura.

A regra principal deste guia: **separe estrutura, aparencia e comportamento**. HTML organiza o conteudo, CSS cuida do visual e JavaScript cuida da interacao. Misturar os tres cedo demais transforma qualquer projeto em um macarrao de codigo.

1. A fundacao do projeto

Antes de escrever telas, crie uma estrutura simples que todos consigam entender.

Estrutura recomendada de pastas

```
sala-do-futuro-2/
|
|-- index.html
|
|-- paginas/
|   |-- aluno.html
|   |-- professor.html
|   |-- atividades.html
|   |-- atividade.html
|   |-- guildas.html
|   |-- ranking.html
|   `-- perfil.html
|
|-- css/
|   |-- geral.css
|   |-- componentes.css
|   `-- paginas.css
|
|-- js/
|   |-- geral.js
|   |-- atividades.js
|   |-- guildas.js
|   `-- ranking.js
|
|-- assets/
|   |-- imagens/
|   |-- icones/
|   `-- logos/
|
`-- dados/
    `-- exemplo.json
```

Nao precisa criar todas as pastas no primeiro dia. Comece com index.html, css/, js/ e assets/. As outras entram quando houver necessidade.

O que vai em cada lugar

Pasta/arquivo	Funcao
index.html	Entrada principal do site.
paginas/	Outras telas HTML.
css/	Estilos visuais.
js/	Logica e interacoes.
assets/imagens/	Fotos, ilustracoes e fundos.
assets/icones/	Icones pequenos.
assets/logos/	Logos do projeto.
dados/	Dados simulados, por exemplo JSON.

2. Ordem de escrita de uma pagina HTML

Uma pagina bem organizada fica previsivel. Isso e bom, apesar de humanos parecerem gostar de esconder coisas aleatoriamente.

```
<!DOCTYPE html>
<html lang="pt-BR">

<head>
  <!-- 1. Configuracoes basicas -->
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <!-- 2. Titulo da pagina -->
  <title>Sala do Futuro 2 - Inicio</title>

  <!-- 3. CSS -->
  <link rel="stylesheet" href="css/geral.css">
</head>

<body>

  <!-- 4. Cabecalho -->
  <header>
    <h1>Sala do Futuro 2</h1>
  </header>

  <!-- 5. Navegacao -->
  <nav>
    <a href="index.html">Inicio</a>
    <a href="paginas/guildas.html">Guildas</a>
  </nav>

  <!-- 6. Conteudo principal -->
  <main>
    <section>
      <h2>Seu progresso</h2>
    </section>
  </main>

  <!-- 7. Rodape -->
  <footer>
    <p>Prototipo escolar</p>
  </footer>

  <!-- 8. JavaScript -->
  <script src="js/geral.js"></script>
</body>
</html>
```

Ordem mental para escrever

- **Head:** configuracoes, titulo e CSS.
- **Header:** identidade da pagina.
- **Nav:** caminhos para outras telas.
- **Main:** conteudo principal.
- **Sections/Articles:** blocos do conteudo.
- **Footer:** informacoes finais.
- **Script:** JavaScript no fim do body. quando possivel.

Colocar o JavaScript perto do fim do <body> ajuda a garantir que os elementos HTML ja existam quando o script rodar.

3. Como vincular o CSS

O HTML chama o arquivo CSS usando a tag link dentro do head.

Quando o HTML esta na raiz

```
<link rel="stylesheet" href="css/geral.css">
```

Quando o HTML esta dentro da pasta paginas

```
<link rel="stylesheet" href="../../css/geral.css">
```

../ significa "voltar uma pasta". Esse detalhe parece pequeno ate voce passar 20 minutos encarando um CSS que "nao funciona" e descobrir que o navegador simplesmente nao achou o arquivo.

Exemplo de CSS inicial

```
/* 1. RESET / BASE */
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Arial, sans-serif;
}

/* 2. ESTRUTURA */
header {
  padding: 20px;
}

main {
  padding: 20px;
}

/* 3. COMPONENTES */
.botao {
  padding: 10px 16px;
}

.card {
  padding: 16px;
}

/* 4. PAGINAS ESPECIFICAS */
.ranking {
  width: 100%;
}
```

Ordem recomendada dentro do CSS

- Reset e configuracoes gerais.
- Variaveis de cores e espacamentos, se usar.
- Body e tipografia.
- Layout principal: header, nav, main, footer.
- Componentes reutilizaveis: botoes, cards, tabelas.
- Estilos de paginas especificas.
- Responsividade no final, com media queries.

4. Como vincular o JavaScript

O HTML chama o JS usando script.

HTML na raiz

```
<script src="js/geral.js"></script>
```

HTML dentro de paginas

```
<script src="../../js/guildas.js"></script>
```

Alternativa: usar defer no head

```
<head>
  <script src="js/geral.js" defer></script>
</head>
```

defer faz o navegador baixar o script sem travar a leitura do HTML e executar depois que a estrutura da pagina estiver pronta.

Exemplo de JS organizado

```
// 1. ELEMENTOS DA PAGINA
const botao = document.querySelector("#botao-entrar");
const mensagem = document.querySelector("#mensagem");

// 2. FUNCOES
function mostrarMensagem() {
  mensagem.textContent = "Bem-vindo ao Sala do Futuro 2!";
}

// 3. EVENTOS
botao.addEventListener("click", mostrarMensagem);
```

Ordem recomendada dentro do JS

- Constantes e referencias aos elementos HTML.
- Dados temporarios ou configuracoes.
- Funcoes pequenas e com nomes claros.
- Eventos: click, submit, change etc.
- Inicializacao da pagina no final.

5. Como HTML, CSS e JS trabalham juntos

HTML: cria o elemento

```
<button id="botao-entrar" class="botao">Entrar</button>
<p id="mensagem"></p>
```

CSS: estiliza pela class

```
.botao {
  border: none;
  padding: 12px 18px;
  cursor: pointer;
}
```

JS: encontra pelo id e adiciona comportamento

```
const botao = document.querySelector("#botao-entrar");
const mensagem = document.querySelector("#mensagem");

botao.addEventListener("click", function () {
  mensagem.textContent = "Entrada simulada!";
});
```

Regra pratica: use class principalmente para estilo e componentes repetidos. Use id para um elemento unico que o JavaScript precisa localizar.

Exemplo de componente reutilizavel

```
<article class="card-atividade">
  <h3>Educacao Financeira</h3>
  <p>Missao: Organize um orcamento.</p>
  <button class="botao" data-atividade="financeira-01">Comecar</button>
</article>
```

Quando varios botoes fazem a mesma coisa, **data-*** e classes ajudam a evitar criar um id diferente para cada um.

6. Organizacao por responsabilidade

Evite um unico arquivo gigante para tudo.

CSS

```
css/
|-- geral.css      -> body, header, nav, fontes, cores
|-- componentes.css -> botoes, cards, tabelas, modais
`-- paginas.css    -> detalhes de telas especificas
```

JavaScript

```
js/
|-- geral.js       -> menu, funcoes gerais
|-- atividades.js  -> questoes, progresso, envio
|-- guildas.js     -> criar, entrar, listar guildas
`-- ranking.js     -> filtros e classificacao
```

Quando criar outro arquivo?

- Quando um arquivo comecar a cuidar de assuntos diferentes demais.
- Quando a logica de uma pagina crescer bastante.
- Quando voce precisar reutilizar uma parte em varias paginas.
- Quando encontrar um trecho e pensar "isso nao tem nada a ver com o resto deste arquivo".

Nao exagere no outro extremo. Criar 47 arquivos para um prototipo de 6 telas tambem e uma arte. Separe por responsabilidade, nao por paranoia.

7. Nomes de arquivos, classes e IDs

Padrao simples evita erros bobos e deixa o projeto legivel.

Arquivos

BOM:

```
ranking-guildas.html  
criar-atividade.html  
atividades.js  
componentes.css
```

EVITE:

```
Ranking Guildas NOVO FINAL.html  
Pagina 2 definitiva real.html  
scriptNOV02.js
```

Classes CSS

```
.card-atividade { }  
.botao-principal { }  
.ranking-guildas { }  
.perfil-aluno { }
```

IDs

```
#form-login  
#botao-entrar  
#lista-guildas  
#xp-atual
```

Regras uteis

- Use minusculas.
- Use hifen em nomes compostos.
- Evite espacos e acentos em nomes de arquivos.
- Diga o que a coisa e, nao como ela parece. Melhor **.botao-principal** do que **.botao-azul**.
- Nao reutilize o mesmo id em varios elementos da mesma pagina.

8. Base de pagina pronta para copiar

Modelo simples para novas telas do projeto.

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sala do Futuro 2 - Pagina</title>

  <link rel="stylesheet" href="../css/geral.css">
  <link rel="stylesheet" href="../css/componentes.css">
  <link rel="stylesheet" href="../css/paginas.css">
</head>
<body>

  <header>
    <h1>Sala do Futuro 2</h1>
  </header>

  <nav>
    <a href="../index.html">Inicio</a>
    <a href="atividades.html">Atividades</a>
    <a href="guildas.html">Guildas</a>
    <a href="ranking.html">Ranking</a>
  </nav>

  <main>
    <section>
      <h2>Titulo da pagina</h2>
      <p>Conteudo aqui.</p>
    </section>
  </main>

  <footer>
    <p>Prototipo Sala do Futuro 2</p>
  </footer>

  <script src="../js/geral.js"></script>
</body>
</html>
```

Se a pagina estiver na raiz, retire o ../. Se estiver dentro de outra pasta, ajuste o caminho.

9. Ordem para construir o projeto sem se perder

Etapa	Acao	Objetivo
1	Criar pastas	index.html, paginas, css, js, assets.
2	Montar HTML	So estrutura e conteudo.
3	Testar navegacao	Todos os links devem funcionar.
4	Criar CSS geral	Fonte, cores, espacamento e layout.
5	Criar componentes	Cards, botoes, tabelas, formularios.
6	Adicionar JavaScript	Interacoes pequenas e testaveis.
7	Criar dados simulados	XP, guildas, atividades e ranking.
8	Integrar backend depois	Somente quando o prototipo visual estiver firme.

Nao pule direto para o backend

Para o Sala do Futuro 2, o melhor e primeiro provar o fluxo: **login simulado -> atividade -> resposta -> XP -> ranking -> guilda**. Depois voces ligam isso a um backend real. Se tentarem resolver servidor, banco, IA, login e interface ao mesmo tempo, os dois devs vao herdar um pequeno incendio.

10. Exemplo de organizacao do Sala do Futuro 2

```
sala-do-futuro-2/
|
|-- index.html           # login / entrada
|
|-- paginas/
|   |-- painel-aluno.html # XP, atividades, guilda
|   |-- painel-professor.html # criar atividade, resultados
|   |-- atividade.html    # responder atividade
|   |-- guildas.html      # criar/entrar/listar
|   |-- ranking.html      # ranking aluno/guildas
|   |-- perfil.html       # perfil e progresso
|
|-- css/
|   |-- geral.css         # base visual
|   |-- componentes.css  # cards, botoes, tabelas
|   |-- paginas.css      # ajustes especificos
|
|-- js/
|   |-- geral.js          # funcoes comuns
|   |-- atividades.js     # atividade e XP
|   |-- guildas.js        # regras de guilda
|   |-- ranking.js        # classificacoes
|
|-- assets/
|   |-- logos/
|   |-- imagens/
|   |-- icones/
|
|-- dados/
|   |-- alunos.json
|   |-- guildas.json
|   |-- atividades.json
```

Uma observacao importante sobre JSON

Arquivos JSON podem servir para **dados simulados** no prototipo. Quando o sistema tiver backend real, esses dados provavelmente vao para banco de dados e rotas da API. Nao use JSON publico para guardar senha, token, chave de API ou dado sensivel.

11. Erros que mais quebram projetos iniciantes

- **Caminho errado:** usar css/geral.css em uma pagina que esta dentro de paginas/.
- **Nome diferente:** chamar ranking.html no link quando o arquivo se chama Ranking.html.
- **JS antes do HTML:** o script procura um elemento que ainda nao foi carregado.
- **IDs duplicados:** dois elementos com o mesmo id.
- **Tudo em um arquivo:** 1 HTML, 1 CSS e 1 JS gigantes fazendo assuntos demais.
- **Copiar e colar sem entender:** o projeto cresce rapido, mas a primeira manutencao vira arqueologia.
- **Chave de API no frontend:** qualquer usuario consegue visualizar. Nunca coloque segredo real em HTML ou JS publico.
- **Nomes caoticos:** final, final2, final-agora-vai, final-real. Um patrimonio cultural da programacao que deve ser evitado.

Checklist rapido

- ☐ Pastas separadas por funcao.
- ☐ Todos os links HTML funcionam.
- ☐ CSS carregando em cada pagina.
- ☐ JS carregando sem erro no console.
- ☐ Classes e IDs com nomes claros.
- ☐ Sem chaves secretas no frontend.
- ☐ Uma funcao por responsabilidade sempre que possivel.
- ☐ Projeto abre a partir do index.html.

12. Resumo de bolso

HTML = estrutura. Ele define o que existe na tela.

CSS = visual. Ele define como a tela parece.

JavaScript = comportamento. Ele define o que acontece quando o usuário interage.

HTML chama CSS:

```
<link rel="stylesheet" href="css/geral.css">
```

HTML chama JS:

```
<script src="js/geral.js"></script>
```

Página dentro de uma pasta volta uma pasta com ../:

```
<link rel="stylesheet" href="../css/geral.css">
```

```
<script src="../js/geral.js"></script>
```

Fluxo ideal para vocês

1. Montar as telas em HTML.
2. Organizar o visual no CSS.
3. Adicionar interações pequenas em JS.
4. Testar tudo.
5. So depois ligar backend, banco de dados e IA.

Essa ordem parece mais lenta no primeiro dia e é muito mais rápida na semana seguinte, quando vocês ainda conseguem entender o próprio código. Uma raridade preciosa.